

جلسه ۸ – توابع

چرا باید از توابع استفاده کنیم؟

توابع بخش بسیار مهمی از برنامه‌نویسی هستند. مهم‌ترین دلایل استفاده از توابع:

- **جلوگیری از تکرار کد:** به جای اینکه یک بخش کد را چند بار بنویسیم، می‌توانیم آن را در قالب یک تابع تعریف کنیم و بارها صدا بزنیم.
- **سازماندهی و خوانایی بیشتر:** کد به بخش‌های کوچک‌تر و قابل‌فهم‌تر تقسیم می‌شود.
- **قابلیت استفاده مجدد (Reusability):** یک تابع یک بار نوشته می‌شود ولی در بخش‌های مختلف برنامه استفاده می‌شود.
- **تقسیم پروژه بزرگ به بخش‌های کوچک:** هر بخش می‌تواند یک وظیفه خاص انجام دهد.

تعریف تابع با def

برای تعریف یک تابع از کلمه کلیدی `def` استفاده می‌کنیم:

```
In [1]: def salam():  
        print("سلام! این یک تابع ساده است")  
  
        salam()
```

سلام! این یک تابع ساده است

پارامتر و مقدار بازگشتی (return)

تفاوت مهم بین `print` و `return` این است که:

- `print` فقط نتیجه را روی صفحه نمایش می‌دهد.
- `return` نتیجه را به برنامه برمی‌گرداند تا بتوانیم دوباره از آن استفاده کنیم.

```
In [2]: def jam(a, b):  
        return a + b  
  
        natije = jam(5, 7)  
        print("نتیجه:", natije)  
        print("نتیجه × 2:", natije * 2)
```

نتیجه: 12

نتیجه × 2: 24

محدوده متغیرها (global و local)

- متغیرهای تعریف شده داخل تابع محلی (local) هستند.
- متغیرهای تعریف شده بیرون از تابع سراسری (global) هستند.

```
In [3]: x = 10 # متغیر global

def test():
    y = 5 # متغیر local
    print("x داخل تابع:", x)
    print("y داخل تابع:", y)

test()
print("x بیرون تابع:", x)
# print(y) # محلی است y خطا می‌دهد چون
```

x 10 : داخل تابع
y 5 : داخل تابع
x 10 : بیرون تابع

پارامترهای پیش فرض

اگر مقدار پیش فرضی برای پارامتر تعریف کنیم، هنگام صدا زدن تابع می‌توانیم آن پارامتر را وارد نکنیم.

```
In [4]: def pow_with_default(base, exp=2):
        return base ** exp

print(pow_with_default(5))      # 25
print(pow_with_default(2, 3))  # 8
```

25
8

تعداد نامحدود آرگومان‌ها (*args, **kwargs)

- `*args`: گرفتن ورودی نامحدود به صورت لیست
- `**kwargs`: گرفتن ورودی نامحدود به صورت دیکشنری

```
In [5]: def jam_var_args(*args):
        return sum(args)

print(jam_var_args(1, 2, 3, 4, 5))

def show_info(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

show_info(name="Ali", age=25, city="Tehran")
```

15
name: Ali
age: 25
city: Tehran

توابع بازگشتی (Recursive Functions)

گاهی یک تابع خودش را صدا می‌زند. این روش برای مسائلی مثل محاسبه فاکتوریل یا دنباله فیبوناچی بسیار مفید است.

- نکته مهم: باید همیشه **شرط پایان** داشته باشیم تا بی‌نهایت صدا زدن رخ ندهد.

```
In [6]: def factorial(n):
        if n == 0 or n == 1:
            return 1
        return n * factorial(n-1)

        print("فاکتوریل 5:", factorial(5))

        def fibonacci(n):
            if n <= 1:
                return n
            return fibonacci(n-1) + fibonacci(n-2)

        print("فیبوناچی 6:", fibonacci(6))
```

فاکتوریل 5: 120

فیبوناچی 6: 8

نکات تکمیلی

داک استرینگ (Docstring)

برای توضیح عملکرد تابع:

```
In [7]: def circle_area(r):
        """را برمی‌گرداند r این تابع مساحت دایره با شعاع"""
        import math
        return math.pi * r * r

        print(circle_area.__doc__)
        print(circle_area(3))
```

را برمی‌گرداند r این تابع مساحت دایره با شعاع

28.274333882308138

Type Hints

برای مشخص کردن نوع ورودی و خروجی توابع:

```
In [8]: def add_numbers(a: int, b: int) -> int:
        return a + b

        print(add_numbers(3, 7))
```

تمرین: محاسبه مساحت و محیط اشکال مختلف

- دایره
- مربع
- مستطیل
- مثلث

```
In [9]: import math

def circle_perimeter(r):
    return 2 * math.pi * r

def square_area(a):
    return a * a

def square_perimeter(a):
    return 4 * a

def rectangle_area(a, b):
    return a * b

def rectangle_perimeter(a, b):
    return 2 * (a + b)

def triangle_area(base, height):
    return 0.5 * base * height

def triangle_perimeter(a, b, c):
    return a + b + c

print("مساحت دایره با شعاع 3:", circle_area(3))
print("محیط مربع با ضلع 4:", square_perimeter(4))
```

مساحت دایره با شعاع 3: 28.274333882308138
محیط مربع با ضلع 4: 16

تمرین‌های اضافه

1. تابعی بنویسید که لیستی از اعداد بگیرد و میانگین آن‌ها را برگرداند.
2. تابعی بنویسید که یک رشته بگیرد و تعداد حروف صدادار (a, i, o, u, e) آن را حساب کند.
3. تابعی بازگشتی بنویسید که عدد ورودی را به توان دیگری برساند.
4. تابعی بنویسید که بررسی کند عدد ورودی اول است یا نه.
5. تابعی بنویسید که بزرگ‌ترین عدد در یک لیست را پیدا کند.
6. تابعی بنویسید که ورودی را به صورت نامحدود (args*) بگیرد و کوچک‌ترین مقدار را برگرداند.